Enterprise FinTech Data Platform (EFTP)

Project Report

Use Case 12 - Enterprise Data Engineering Capstone L&T EduTech

Prepared by: Abhay Yadav (26MAS1005)

Gayathri Nandakumar (26MAS1011)

P. Akhil Kumar Reddy (26MAS1017)

GitHub: https://github.com/abhayadav-list/EFTP

1. Executive Summary

ABC FinTech Solutions Ltd. runs UPI, wallet, gateway and settlement systems that operate independently, so it cannot reliably monitor success rates, detect fraud, reconcile settlements or see customer behaviour in one place. The Enterprise FinTech Data Platform (EFTP) solves this by integrating ten source systems in five formats (CSV, Excel, JSON, XML, SQL) into a PostgreSQL data warehouse organised as bronze (staging), silver (cleansed) and gold (star schema) layers.

Pentaho Data Integration performs ingestion and warehouse loading, Python performs profiling and cleansing, and Power BI delivers the executive dashboards. Metadata, lineage, audit logging and reconciliation provide governance. Git and GitHub hold every artefact.

Headline results from the project runs:

- 11,905 cleansed transactions are in the fact table: UPI 6,634, wallet 2,900, gateway 2,371.
- Six categories of data quality issue were measured and resolved, and every rejected row was kept for audit.
- All ten sources are documented in an inventory, data dictionary, source-to-target mapping and lineage.
- A real-time Kafka ingestion path was added as an extension beyond the brief.

Deliverable index

Sprint	Deliverable named in the brief	Section	Content

Sprint	Deliverable named in the brief	Section	Content
0	Business Requirement Document	2	Written
0	Project Charter	3	Written; fill names and dates
0	Solution Architecture	4	Written
0	Product Backlog and Sprint Backlog	5	Written; update statuses
0	Git Repository	6	Written; add link
1	Source Inventory	7	Written
1	Data Dictionary	8	Written
1	Pentaho ETL Pipelines	9	Written; screenshots
1	Staging Database	10	Written; counts query
1	Git Commit History	6	Commands provided
2	Data Profiling Report	11	Written; profiling query
2	Data Quality Report	12	Written; verification queries
2	Star Schema	13	Written
2	PostgreSQL Data Warehouse	14	Written; verification queries
2	SQL Scripts	15	Written
2	Git Repository Updates	6	Commands provided
3	Source-to-Target Mapping	16	Written; export query
3	Data Lineage Document	17	Written
3	Metadata Repository	18	Written; population queries
3	FinTech Business Glossary	19	Written

Sprint	Deliverable named in the brief	Section	Content
3	Pentaho Jobs	20	Written; screenshots
3	Validation Report	21	Method and queries; paste results
3	Power BI Dashboard	22	Design and measures; screenshots
3	Final Git Release	6	Commands provided
3	Project Report	This document	
3	Sprint Presentation	25	Outline
Brief	Definition of Done	24	Checklist
Brief	Learning Outcomes	27	Mapping
Extension	Real-time ingestion	23	Written

2. Business Requirement Document (Sprint 0)

2.1 Business problem

Transaction data is produced by ten operational systems that do not share formats, keys or timing. The organisation therefore cannot monitor success and failure rates, detect payment failures and fraud, reconcile settlements against transactions, or understand customer payment behaviour from one trusted source.

2.2 Objectives

1. Collect financial transaction data from multiple heterogeneous systems.
2. Clean, validate and standardise payment and customer data.
3. Store curated datasets in a PostgreSQL data warehouse.
4. Maintain metadata, audit logs and end-to-end data lineage.
5. Support payment analytics, fraud monitoring, settlement reconciliation and executive reporting.
6. Use Git and GitHub for collaborative version control.
7. Produce enterprise-standard documentation and sprint deliverables.

2.3 Stakeholders

Stakeholder	Interest	Needs from the platform

Stakeholder	Interest	Needs from the platform
Executive leadership	Business performance	Dashboard of volume, value and success rate
Payments operations	Reliability	Success and failure rates by channel
Fraud and risk	Loss prevention	Fraud scores, flagged transactions, trends
Finance and settlement	Accurate payouts	Reconciliation of transactions against settlements
Merchant management	Merchant health	Merchant and category performance
Compliance and KYC	Regulation	KYC status, PII identified, audit trail
Data engineering	Maintainability	Rerunnable pipelines, documentation, lineage
BI and analytics	Insight	A clean star schema to report from

2.3.1 Scope

In scope: ingestion of ten sources in five formats; staging, cleansing and a star-schema warehouse; profiling and quality reporting; metadata, lineage and glossary; Pentaho orchestration with audit logging; settlement reconciliation; Power BI dashboards; Git release; a real-time ingestion extension.

Out of scope: production deployment, real customer data, training a fraud model, true event-at-a-time stream processing, and slowly changing dimension history (Type 2).

2.4 Requirements

ID	Functional requirement	Sprint
FR-01	Ingest CSV, Excel, JSON, XML and SQL sources into staging tables	1
FR-02	Keep raw data unchanged, with audit columns source_system, batch_id and load_timestamp	1
FR-03	Profile every staged dataset	2
FR-04	Detect duplicate transactions, missing customers, invalid	2

ID	Functional requirement	Sprint
	statuses and inconsistent merchant data	
FR-05	Cleanse and standardise data, keeping rejected rows with reasons	2
FR-06	Load a star schema of fact and dimension tables	2
FR-07	Provide SQL reporting queries for business questions	2
FR-08	Document mappings, lineage, glossary and a metadata catalog	3
FR-09	Orchestrate the ETL with Pentaho jobs and log every run	3
FR-10	Validate settlement reconciliation	3
FR-11	Deliver executive dashboards in Power BI	3

ID	Non-functional requirement
NFR-01	Rerunnable: any load can be repeated without duplicating data
NFR-02	Traceable: any gold value can be traced to its source
NFR-03	Auditable: rejected rows and job runs are retained
NFR-04	Privacy: PII columns are identified in the metadata catalog
NFR-05	Versioned: every deliverable is committed to Git

2.5 Key performance indicators

KPI	Definition	Source
Transaction success rate	Successful transactions / all	fact_transactions

KPI	Definition	Source
	transactions	
Failure rate	Failed transactions / all transactions	fact_transactions
Total transaction value	Sum of amounts of successful transactions	fact_transactions
Fraud rate (scored)	Flagged / fraud-scored transactions	fact_transactions
Settlement shortfall %	(total_amount - settled_amount) / total_amount	fact_settlement
Reconciliation match rate	Merchant-weeks within tolerance / merchant-weeks	reconciliation_result
Rejection rate	Rejected rows / staged rows	silver/validation files

2.6 Assumptions and constraints

- **The data is synthetic, generated to mirror the ten systems with shared keys (customer_id, merchant_id, transaction_id).** The period is January to June 2025 and amounts are in rupees.
- Settlement batches are weekly (Monday to Sunday) and dated on the Sunday.
- The payment gateway source carries no customer identifier.
- Only about 8% of transactions are fraud-scored.
- The environment is a single local machine.

2.7 Risks and mitigations

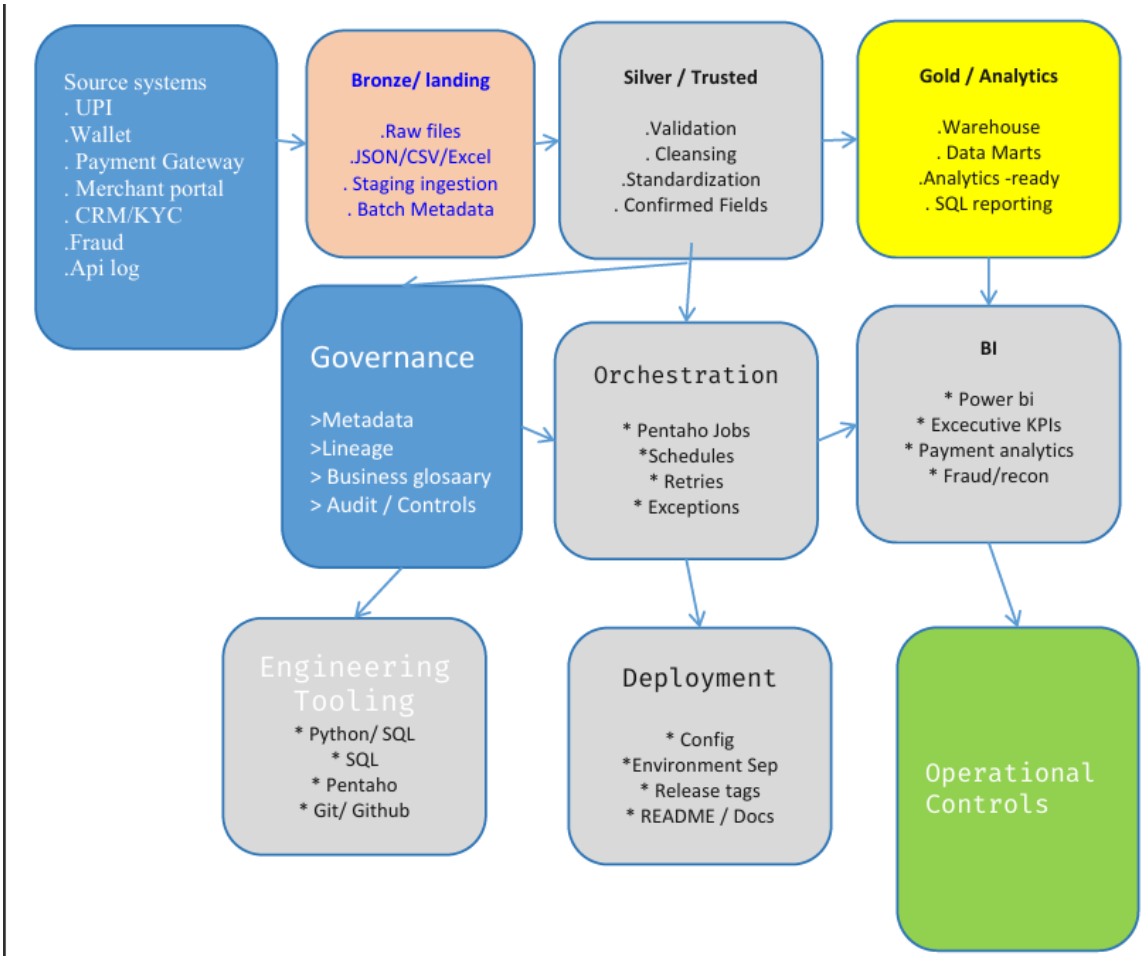
Risk	Mitigation
Formats and types differ across sources	Raw text staging; one Pentaho transformation per format
Orphaned or missing keys	LEFT JOINs and NULL surrogate keys, documented rather than hidden
Tool and version incompatibility (Python 3.13 with ydata-profiling)	Virtual environment and a pandas fallback for profiling
Reconciliation misread because of week boundaries	Documented week rule and a two-level comparison
Reruns duplicating data	Truncate-and-load and unique constraints

3. Project Charter

Item	Detail
Project	UC12 Enterprise FinTech Data Platform (EFTP)
Organisation	ABC FinTech Solutions Ltd. (fictional), via L&T EduTech capstone
Method	Agile Scrum, four sprints (0 to 3), sprint reviews at the end of each
Purpose	Build a unified, governed payments analytics platform
Goals	Ingest ten sources; cleanse and model them; reconcile settlements; report to executives
Success criteria	Definition of Done met every sprint (section 24)
Constraints	Pentaho, PostgreSQL, Python (Pandas), Git and GitHub, Power BI

Sprint	Focus	Digital assignment
0	Initiation and architecture	DA 1
1	Data discovery and ingestion	DA 1
2	Data profiling and data warehouse	DA 2
3	Governance, lineage and analytics	DA 3

4. Layered architecture



4.2 Technology stack

Category	Technology	Use in this project
ETL	Pentaho Data Integration (Spoon) 9.4	Ingestion, dimension and fact loads, jobs
Database	PostgreSQL	staging, silver, gold and metadata schemas
Programming	Python (Pandas)	Profiling, cleansing, tests
Version control	Git and GitHub	Source and documents
Reporting	Power BI	Executive dashboards
Documentation	Markdown and MS Word	Project documents
Project management	Agile Scrum	Sprints and backlog

Category	Technology	Use in this project
Streaming extension	Apache Kafka (Docker), kafka-python	Real-time ingestion

4.3 Design principles

- Raw data is preserved in bronze and typed only in silver.
- Surrogate keys in the warehouse; natural keys kept for matching.
- Every load is rerunnable.
- No silent row loss: LEFT JOINs, followed by counts and NULL checks.
- Rejected rows are written out with a reason, never discarded.

4.4 Repository structure

```
Project/
  docs/  config/  datasets/
  bronze/ (ingestion, staging)
  silver/ (cleansing, validation, transformations)
  gold/   (warehouse, datamarts, analytics)
  pentaho/ (transformations, jobs)
  python/  sql/  metadata/  lineage/  dashboards/  deployment/  tests/
  README.md
```

5. Product Backlog and Sprint Backlog

ID	User story	Sprint	Priority	Status
US-01	Source inventory and data dictionary so every source is understood	0-1	High	Done
US-02	Ingest CSV sources to staging	1	High	Done
US-03	Ingest Excel, JSON, XML and SQL sources to staging	1	High	Done
US-04	Staging tables with audit columns	1	High	Done
US-05	Basic logging and exception handling	1	Medium	Done

ID	User story	Sprint	Priority	Status
US-06	Profile all staged datasets	2	High	Done
US-07	Quantify duplicates, missing customers, invalid statuses, merchant inconsistency	2	High	Done
US-08	Cleanse data and keep rejected rows	2	High	Done
US-09	Design the star schema	2	High	Done
US-10	Load dimension tables	2	High	Done
US-11	Load fact tables	2	High	Done
US-12	SQL queries for business reporting	2	Medium	Done
US-13	Source-to-target mapping	3	High	Done
US-14	Data lineage document	3	High	Done
US-15	Business glossary	3	Medium	Done
US-16	Metadata repository	3	High	Done
US-17	Master Pentaho job with audit logging	3	High	Done
US-18	Settlement reconciliation validation	3	High	Done
US-19	Power BI executive	3	High	Done

ID	User story	Sprint	Priority	Status
	dashboards			
US-20	Final release tag, project report, presentation	3	High	Done
US-21	Real-time ingestion extension	Extra	Could	Done

6. Git Repository

Repository: https://github.com/abhayadav-list/EFTP

7. Source Inventory (Sprint 1)

#	Source system	Format	File or object	Volume	Staging table
1	UPI Transaction System	CSV	upi_transactions.csv	6,760 rows	staging.upi_transactions
2	Digital Wallet Platform	CSV	digital_wallet_transactions.csv	2,954 rows	staging.digital_wallet_transactions
3	Payment Gateway	JSON	payment_gateway_transactions.json	2,406 records	staging.payment_gateway_transactions
4	Merchant Portal	Excel	merchant_portal.xlsx (sheet Merchants)	150 rows	staging.merchant_portal
5	Customer Relationship Management	SQL (SQLite)	crm_customers.db (table customers)	1,980 rows	staging.crm_customers
6	KYC Verification System	JSON	kyc_verification.json	2,000 records	staging.kyc_verification
7	Settlement Processing System	CSV	settlement_batches.csv	3,864 rows	staging.settlement_batches

#	Source system	Format	File or object	Volume	Staging table
8	Mobile Banking Application	JSON	mobile_banking_sessions.json	6,000 records	staging.mobile_banking_sessions
9	Fraud Detection System	CSV	fraud_detection_flags.csv	970 rows	staging.fraud_detection_flags
10	API Gateway Logs	XML	api_gateway_logs.xml (path /logs/entry)	5,200 entries	staging.api_gateway_logs
Ext.	Real-time event stream	JSON over Kafka	topic eftp.transactions	Continuous	staging.stream_events

Total raw records in the ten batch sources: 32,284. Format coverage: CSV 4, JSON 3, Excel 1, SQL 1, XML 1, so all five formats named in the brief are represented.

Source	Business purpose	Key fields	Joins to	Known data notes
UPI	UPI payments by customers	transaction_id, customer_id, merchant_id, amount, status	CRM, Merchant Portal	Null amounts, status casing varies
Wallet	Wallet load, spend and peer-to-peer transfers	wallet_txn_id, transaction_id, txn_type	CRM, Merchant Portal	merchant_id null for P2P transfers
Gateway	Merchant payments through the gateway	gateway_txn_id, transaction_id, payment_mode	Merchant Portal	No customer_id in the source
Merchant Portal	Merchant master data	merchant_id, category, settlement_cycle	All transaction sources, Settlement	About 15% of names have casing variants
CRM	Customer master data	customer_id, segment, city, state	KYC, transaction sources	20 customers referenced elsewhere are missing
KYC	Identity verification and risk	customer_id, kyc_status, risk_score	CRM	verification_date null while PENDING

Source	Business purpose	Key fields	Joins to	Known data notes
Settlement	Weekly merchant payout batches	settlement_id, merchant_id, total_amount, settled_amount	Merchant Portal	Settled amount is 0 to 3% below total
Mobile Banking	App sessions	session_id, customer_id, transaction_id	CRM, transactions	About 15% of sessions have no transaction
Fraud Detection	Fraud scores on a sample	transaction_id, fraud_score, is_fraud	Transaction sources	Only about 8% of transactions scored
API Gateway Logs	Technical audit trail of API calls	request_id, transaction_id, endpoint, status_code	Transaction sources	Health checks have no transaction_id

8. Data Dictionary (Sprint 1)

Every staging column is TEXT. Three audit columns are added to every staging table: source_system, batch_id and load_timestamp. In staging, the timestamp column of the three transaction sources is named txn_timestamp, and in API logs it is log_timestamp. Sample values and null counts for every field are in the companion file Data_Dictionary.md.

Source	Fields	Keys and notes
UPI	transaction_id, customer_id, merchant_id, upi_vpa, amount (decimal), timestamp (datetime), status, bank_rrn, upi_app	PK transaction_id; FK customer_id to CRM and KYC; FK merchant_id to Merchant Portal
Wallet	wallet_txn_id, transaction_id, customer_id, merchant_id, wallet_provider, txn_type, amount, wallet_balance_after, timestamp, status	PK wallet_txn_id; txn_type is LOAD, SPEND or P2P_TRANSFER
Gateway	gateway_txn_id, transaction_id, merchant_id, amount, payment_mode, gateway_status,	PK gateway_txn_id; status column is gateway_status

Source	Fields	Keys and notes
	response_code, timestamp	
Merchant Portal	merchant_id, merchant_name, category, city, state, onboarding_date, gst_number, bank_account, settlement_cycle	PK merchant_id; settlement_cycle is T+1, T+2 or Weekly
CRM	customer_id, name, gender, dob, phone, email, city, state, signup_date, customer_segment, support_tickets_count	PK customer_id; segment is Retail, Premium or Merchant-linked
KYC	customer_id, kyc_status, id_type, verification_date, risk_score	One row per customer; status VERIFIED, PENDING or REJECTED
Settlement	settlement_id, merchant_id, batch_date, total_transactions, total_amount, settled_amount, settlement_status	PK settlement_id; batch_date is the Sunday ending the week
Mobile Banking	session_id, customer_id, transaction_id, device_type, os_version, app_version, login_timestamp, action	PK session_id; transaction_id nullable
Fraud Detection	transaction_id, fraud_score (0 to 1), is_fraud (0 or 1), fraud_reason, model_version, flagged_timestamp	Joins on transaction_id
API Gateway Logs	request_id, transaction_id, endpoint, http_method, status_code, response_time_ms, timestamp, client_ip	PK request_id; transaction_id blank for health checks

Join key	Appears in
transaction_id	UPI, wallet, gateway, mobile banking, fraud, API logs
customer_id	UPI, wallet, CRM, KYC, mobile banking

Join key	Appears in
merchant_id	UPI, wallet, gateway, Merchant Portal, settlement

9. Pentaho ETL Pipelines (Sprint 1 and 2)

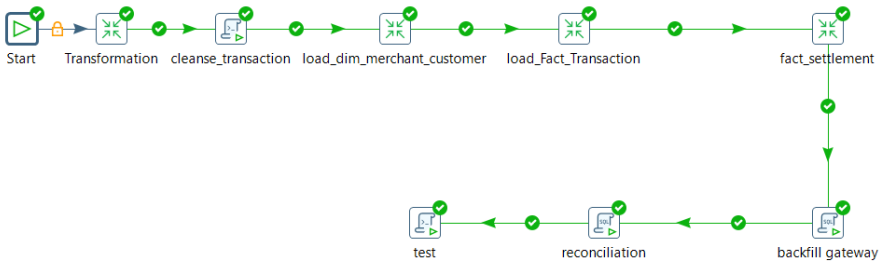
Pipeline	Pattern	Target
Ingest UPI, wallet, settlement, fraud (4 files)	CSV file input, Add constants, Get System Info, Table output	staging tables
Ingest Merchant Portal	Microsoft Excel Input (sheet Merchants), then the same pattern	staging.merchant_portal
Ingest payment gateway, KYC, mobile banking	JSON Input with a JSONPath per field, then the same pattern	staging tables
Ingest API gateway logs	Get Data From XML with loop path /logs/entry, then the same pattern	staging.api_gateway_logs
Ingest CRM	Table Input over an SQLite connection (or CSV export), then the same pattern	staging.crm_customers
load_dim	Table Input from silver, Dimension Lookup/Update (Type 1, Update on every attribute)	gold.dim_customer, gold.dim_merchant
load_fact_transactions	Table Input with surrogate-key joins, Table output with truncate	gold.fact_transactions
load_fact_settlement	Table Input with surrogate-key joins, Table output with truncate	gold.fact_settlement
stream_events ingestion and sub-transformation kafkarule	Kafka Consumer, Get records from stream, Table output	staging.stream_events

Audit constants. Each ingestion adds source_system (UPI_TXN, DIGITAL_WALLET, PAYMENT_GATEWAY, MERCHANT_PORTAL, CRM, KYC_VERIFICATION, SETTLEMENT_PROCESSING, MOBILE_BANKING, FRAUD_DETECTION, API_GATEWAY_LOGS), a batch_id, and a load_timestamp from Get System Info.

Logging and exception handling. Each input step has an error-handling hop to a text file under bronze/staging/errors, which captures the failed row with the error description. This was tested with a

deliberately malformed row. Step metrics (rows read and written) are checked after every run, and the load commit size and batch update are set on every Table output.

Dimension loads. The technical key sequence is entered without the schema prefix, the target schema is set separately, and fields that exist only in the table (load_date) are left out of the field mapping so database defaults apply.



10. Staging Database (Sprint 1)

Ten staging tables, plus stream_events for the extension, hold the raw data as TEXT with three audit columns. Loads truncate before insert so a rerun does not duplicate rows.

Query | Query History

```sql
SELECT 'upi_transactions' AS tbl, COUNT(*) AS row_count FROM staging.upi_transactions
UNION ALL SELECT 'digital_wallet_transactions', COUNT(*) FROM staging.digital_wallet_transactions
UNION ALL SELECT 'payment_gateway_transactions', COUNT(*) FROM staging.payment_gateway_transactions
UNION ALL SELECT 'merchant_portal', COUNT(*) FROM staging.merchant_portal
UNION ALL SELECT 'crm_customers', COUNT(*) FROM staging.crm_customers
UNION ALL SELECT 'kyc_verification', COUNT(*) FROM staging.kyc_verification
UNION ALL SELECT 'settlement_batches', COUNT(*) FROM staging.settlement_batches
UNION ALL SELECT 'mobile_banking_sessions', COUNT(*) FROM staging.mobile_banking_sessions
UNION ALL SELECT 'fraud_detection_flags', COUNT(*) FROM staging.fraud_detection_flags
UNION ALL SELECT 'api_gateway_logs', COUNT(*) FROM staging.api_gateway_logs
ORDER BY 1;

SELECT table_name, COUNT(*) AS column_count
FROM information_schema.columns WHERE table_schema = 'staging'
GROUP BY table_name ORDER BY table_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10

	tbl text	row_count bigint
1	api_gateway_logs	5200
2	crm_customers	1980
3	digital_wallet_transactions	2954
4	fraud_detection_flags	24250
5	kyc_verification	2000
6	merchant_portal	150
7	mobile_banking_sessions	6000
8	payment_gateway_transactio...	2406
9	settlement_batches	3864
10	upi_transactions	6760

	table_name name	column_count bigint
1	api_gateway_logs	11
2	crm_customers	14
3	digital_wallet_transactions	13
4	fraud_detection_flags	9
5	kyc_verification	8
6	merchant_portal	12
7	mobile_banking_sessions	11
8	payment_gateway_transactio...	11
9	settlement_batches	10
10	stream_events	3
11	upi_transactions	12

11. Data Profiling Report (Sprint 2)

11.1 Method

Python script data_profiling.py loads each staging table with pandas and writes one profile per table to docs/profiling. It uses ydata-profiling when available and falls back to a plain pandas summary otherwise.

11.3 Key findings

Table	Finding
upi_transactions	37 null amounts in the source; 9 status spellings (3 statuses in 3 casings); 69 duplicated transaction IDs
digital_wallet_transactions	merchant_id empty on peer-to-peer transfers (899 rows in the source); 13 null amounts; 32 duplicated IDs
payment_gateway_transactions	11 null amounts; 19 duplicated IDs; no customer_id column; status column named gateway_status
merchant_portal	About 15% of merchant names carry upper or lower case variants
crm_customers	1,980 rows against 2,000 customers in KYC, so 20 customers are missing
kyc_verification	verification_date null for the 208 PENDING records
settlement_batches	settled_amount is 0 to 3% below total_amount on every batch
mobile_banking_sessions	926 of 6,000 sessions have no transaction_id (browsing only)
fraud_detection_flags	970 scored rows; fraud_reason is populated for only 2, so confirmed fraud is rare in this data
api_gateway_logs	714 of 5,200 entries have no transaction_id (health checks and pre-transaction calls)

12. Data Quality Report (Sprint 2)

12.1 Issues found and resolved

Issue	Table	Found	Resolution
Duplicate transaction_id	UPI, wallet, gateway	69, 32 and 19 duplicated IDs (138, 64 and 38 rows)	First occurrence kept; the rest written to the rejected-rows file
Missing customer records	CRM against transactions	20 customer IDs (1.01% of 1,984)	Rows kept; customer_sk left NULL
Inconsistent payment status	UPI, wallet, gateway	9 spellings per table	Standardised to UPPERCASE
Null amounts	UPI, wallet, gateway	37, 13 and 11	Rejected with reason NULL_AMOUNT
Negative amounts	UPI, wallet, gateway	21, 10 and 5	Rejected with reason NEGATIVE_AMOUNT
Inconsistent merchant naming	merchant_portal	86 of 150 differ from Title Case; about 15% are true casing variants	Standardised to Title Case
Settlement variance	settlement_batches	3,385 of 3,864 differ by more than 1 rupee	Reclassified by percentage shortfall (section 21)
Gateway rows without a customer	payment gateway	All 2,371	942 recovered from mobile banking sessions; 1,429 remain NULL

12.2 Row counts before and after cleansing

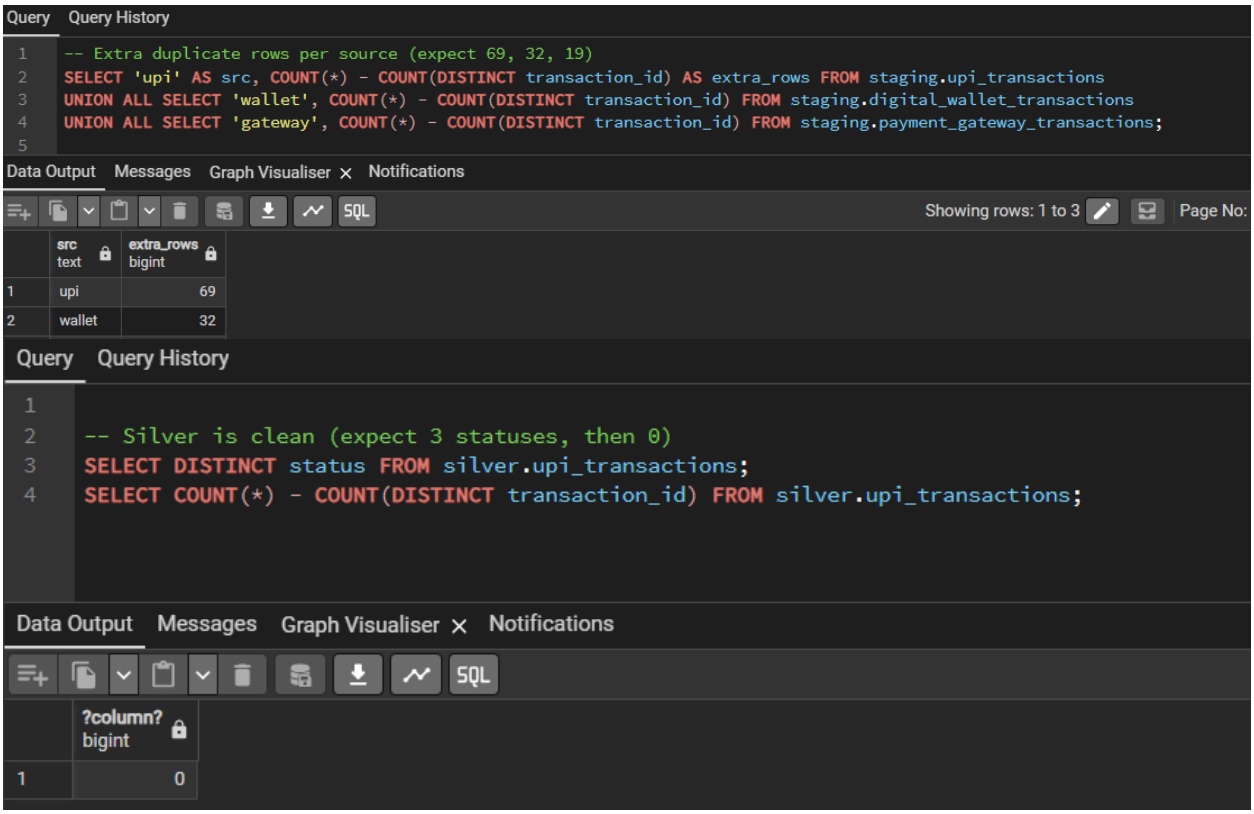
Table	Staged	Rejected	Silver
UPI transactions	6,760	126	6,634
Wallet transactions	2,954	54	2,900
Gateway transactions	2,406	35	2,371
Total in the fact table			11,905

Rejected rows are kept in silver/validation as one CSV per source, each with a reject_reason column.

12.3 Interpretation

- The merchant check compares names with Python's Title Case, so legal suffixes such as LLC are also flagged. The true inconsistency is the all-uppercase and all-lowercase names, which query 12.4 counts exactly.
- Almost every settlement batch is flagged at a one rupee tolerance because every payout is 0 to 3% below the total. A percentage threshold is the meaningful test.
- After the backfill, customer_sk is NULL for 67 UPI rows, 35 wallet rows and 1,429 gateway rows. The UPI and wallet rows are the orphaned customers. The gateway rows are unattributed because the source has no customer field, which is 12.0% of all transactions.

12.4 Verification queries



Query | Query History

```sql
-- Customers in transactions but missing from CRM (expect 20)
SELECT COUNT(DISTINCT t.customer_id) AS missing_customers
FROM (SELECT customer_id FROM staging.upi_transactions
      UNION SELECT customer_id FROM staging.digital_wallet_transactions) t
LEFT JOIN staging.crm_customers c ON c.customer_id = t.customer_id
WHERE c.customer_id IS NULL;
```

Data Output | Messages | Graph Visualiser | Notifications

	missing_customers bigint
1	20

Query | Query History

```sql
-- Status spellings
SELECT status, COUNT(*) FROM staging.upi_transactions GROUP BY status ORDER BY status;
```

Data Output | Messages | Graph Visualiser | Notifications

	status text	count bigint
1	failed	200
2	Failed	194
3	FAILED	229
4	pending	63
5	Pending	76
6	PENDIN...	90
7	success	1939
8	Success	1947
9	SUCCE...	2022

Query | Query History

```sql
-- Null and negative amounts (expect 37/21, 13/10, 11/5)
SELECT 'upi' AS src,
  COUNT(*) FILTER (WHERE NULLIF(NULLIF(amount, ''), 'nan') IS NULL) AS null_amounts,
  COUNT(*) FILTER (WHERE amount LIKE '-%') AS negative_amounts
FROM staging.upi_transactions
UNION ALL SELECT 'wallet',
  COUNT(*) FILTER (WHERE NULLIF(NULLIF(amount, ''), 'nan') IS NULL),
  COUNT(*) FILTER (WHERE amount LIKE '-%') FROM staging.digital_wallet_transactions
UNION ALL SELECT 'gateway',
  COUNT(*) FILTER (WHERE NULLIF(NULLIF(amount, ''), 'nan') IS NULL),
  COUNT(*) FILTER (WHERE amount LIKE '-%') FROM staging.payment_gateway_transactions;
```

Data Output | Messages | Graph Visualiser | Notifications

	src text	null_amounts bigint	negative_amounts bigint
1	upi	37	21
2	wallet	13	10
3	gatew...	11	5

13. Star Schema (Sprint 2)

13.1 Grain

- fact_transactions: one row per transaction, across UPI, wallet and gateway channels.
- fact_settlement: one row per merchant per weekly settlement batch. It joins to dim_merchant and dim_date, and is never joined row by row to fact_transactions. Reconciliation aggregates transactions to merchant and week first.

13.2 Diagram

```
                dim_date
                   |
dim_customer -- fact_transactions -- dim_merchant
                   |
                dim_channel

dim_merchant -- fact_settlement -- dim_date
```

13.3 Tables

Table	Type	Source	Columns
fact_transactions	Fact	silver UPI, wallet and gateway tables, plus fraud flags	transaction_sk, transaction_id, customer_sk, merchant_sk, date_sk, channel_sk, amount, status, is_fraud, fraud_score, load_date
fact_settlement	Fact	silver.settlement_batches	settlement_sk, settlement_id, merchant_sk, date_sk, total_amount, settled_amount,

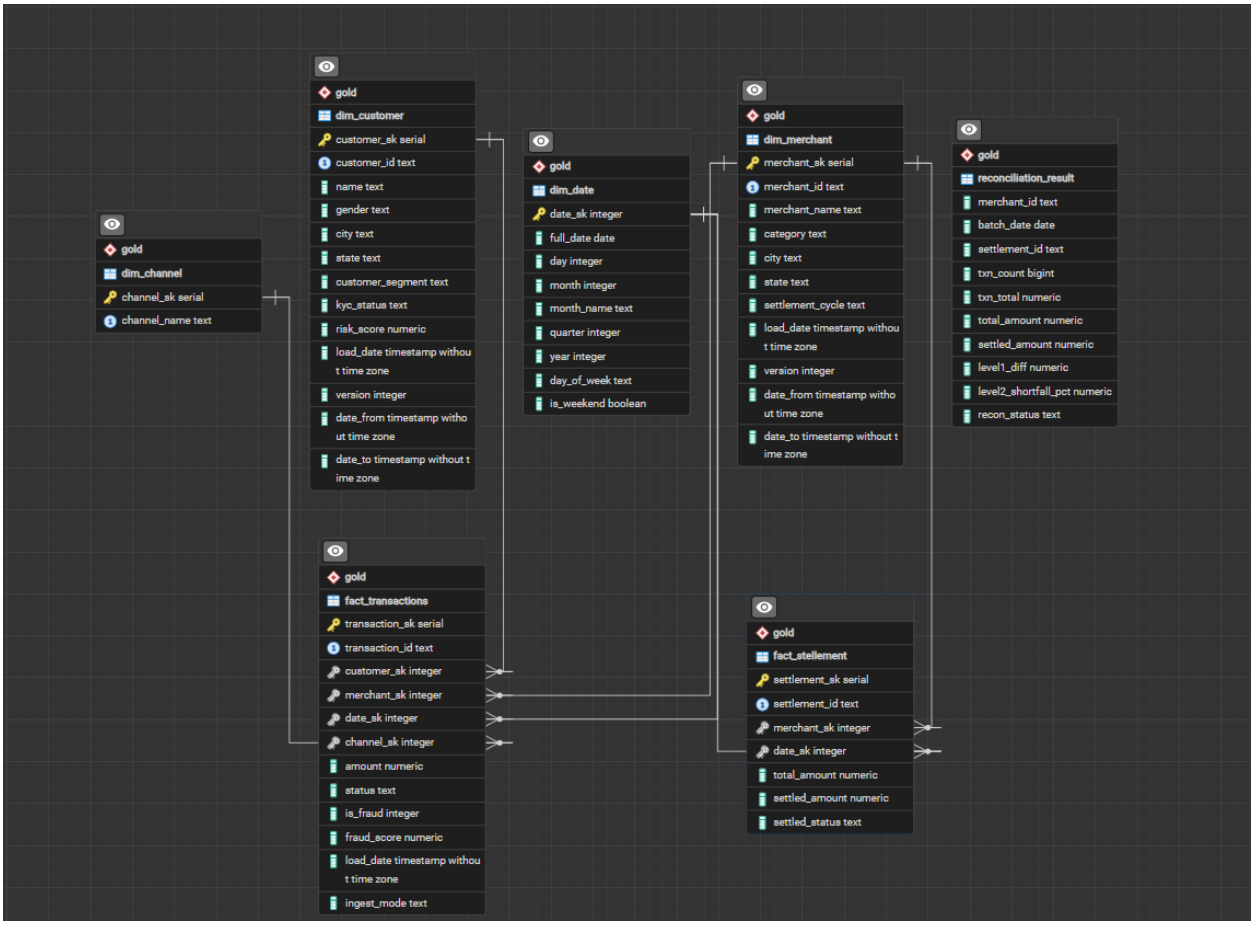
Table	Type	Source	Columns
			settlement_status (total_transactions is optional)
dim_customer	Dimension	CRM left joined to KYC	customer_sk, customer_id, name, gender, city, state, customer_segment, kyc_status, risk_score, load_date
dim_merchant	Dimension	silver.merchant_portal	merchant_sk, merchant_id, merchant_name, category, city, state, settlement_cycle, load_date
dim_date	Dimension	Generated for 2025	date_sk (YYYYMMDD), full_date, day, month, month_name, quarter, year, day_of_week, is_weekend
dim_channel	Dimension	Seeded	channel_sk, channel_name (UPI, WALLET, GATEWAY)

The dimension loader also maintains version, date_from and date_to columns on dim_customer and dim_merchant.

13.4 Design decisions

- One fact table serves all three channels because they share a grain and the same dimensions. channel_sk acts as a slicer, so reports need no UNION.
- merchant_sk is nullable because peer-to-peer wallet transfers have no merchant.
- customer_sk is nullable because the gateway source has no customer identifier and about 20 customers are missing from the CRM.
- Fraud scores are joined with a LEFT JOIN after deduplicating on transaction_id, because only about 8% of transactions are scored. A NULL means not scored.
- Settlement is a separate fact because its grain (merchant and week) differs from a transaction.
- dim_date uses a YYYYMMDD integer key, which is readable in ad hoc queries.
- Dimensions are Type 1 (overwrite on change); history tracking is out of scope.

13.5 Deployed definition



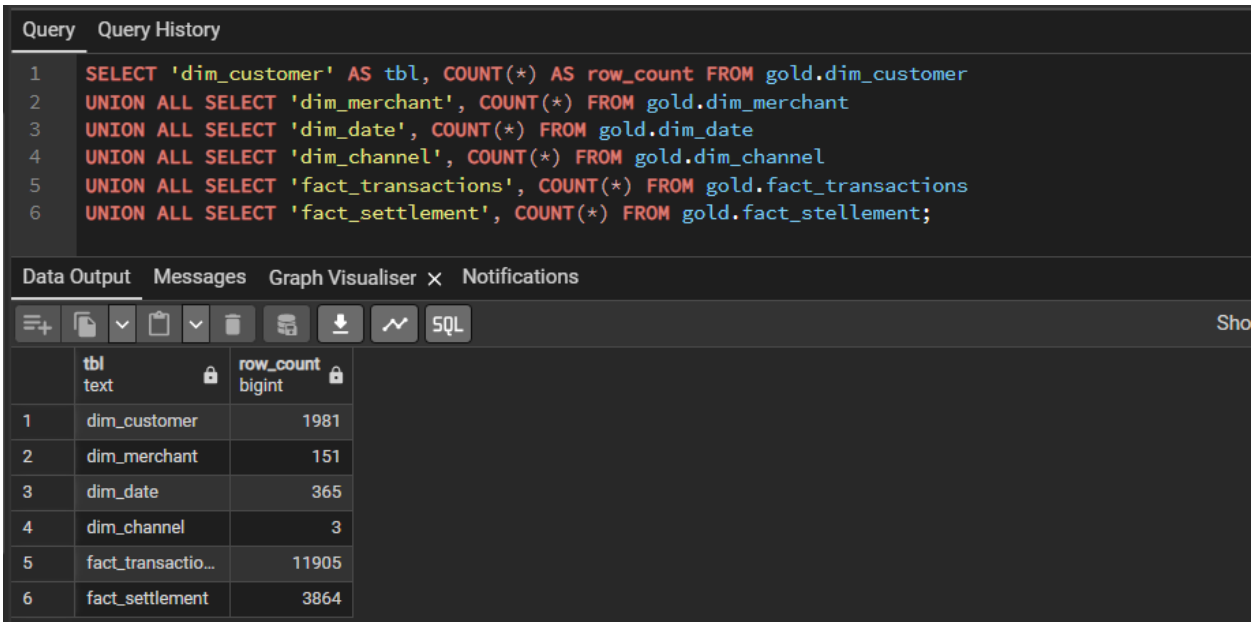
ERD to gold shema

14. PostgreSQL Data Warehouse (Sprint 2)

Schema	Layer	Contents	Loaded by
staging	Bronze	Raw text tables with audit columns	Pentaho ingestion
silver	Silver	Cleansed and deduplicated tables	Python cleansing script
gold	Gold	Star schema, reconciliation_result	Pentaho dimension and fact loads
metadata	Governance	Catalog, lineage map, job log	SQL and the Pentaho job

Load order. staging, then silver, then dimensions, then facts (dimensions first because facts need their surrogate keys), then the optional customer backfill, then reconciliation.

Known results. fact_transactions holds 11,905 rows: UPI 6,634, wallet 2,900 and gateway 2,371. customer_sk is NULL for 67 UPI rows, 35 wallet rows and 1,429 gateway rows.



Query Query History

```
SELECT 'fact rows equal silver rows' AS check_name,
  CASE WHEN (SELECT COUNT(*) FROM gold.fact_transactions) =
            (SELECT COUNT(*) FROM silver.upi_transactions)
          + (SELECT COUNT(*) FROM silver.digital_wallet_transactions)
          + (SELECT COUNT(*) FROM silver.payment_gateway_transactions)
       THEN 'PASS' ELSE 'FAIL' END AS result
UNION ALL SELECT 'no duplicate transaction_id',
  CASE WHEN COUNT(*) = COUNT(DISTINCT transaction_id) THEN 'PASS' ELSE 'FAIL' END
  FROM gold.fact_transactions
UNION ALL SELECT 'every row has date_sk and channel_sk',
  CASE WHEN COUNT(*) FILTER (WHERE date_sk IS NULL OR channel_sk IS NULL) = 0 THEN 'PASS' ELSE 'FAIL' END
  FROM gold.fact_transactions
UNION ALL SELECT 'only valid statuses',
  CASE WHEN COUNT(*) FILTER (WHERE status NOT IN ('SUCCESS', 'FAILED', 'PENDING')) = 0 THEN 'PASS' ELSE 'FAIL' END
  FROM gold.fact_transactions
UNION ALL SELECT 'no null or negative amounts',
  CASE WHEN COUNT(*) FILTER (WHERE amount IS NULL OR amount < 0) = 0 THEN 'PASS' ELSE 'FAIL' END
  FROM gold.fact_transactions
UNION ALL SELECT 'settlement rows equal silver rows',
  CASE WHEN (SELECT COUNT(*) FROM gold.fact_stellement) =
            (SELECT COUNT(*) FROM silver.settlement_batches) THEN 'PASS' ELSE 'FAIL' END;
```

Data Output Messages Graph Visualiser Notifications

Showing rows: 1 to 6 Page No: 1

	check_name text	result text
1	fact rows equal silver rows	PASS
2	no duplicate transaction_id	PASS
3	every row has date_sk and channel_sk	PASS
4	only valid statuses	PASS
5	no null or negative amounts	PASS
6	settlement rows equal silver rows	PASS

Query Query History

```
SELECT ch.channel_name, COUNT(*) AS row_count,
       COUNT(*) FILTER (WHERE f.customer_sk IS NULL) AS null_customer,
       COUNT(*) FILTER (WHERE f.merchant_sk IS NULL) AS null_merchant
FROM gold.fact_transactions f
JOIN gold.dim_channel ch ON ch.channel_sk = f.channel_sk
GROUP BY ch.channel_name;
```

Data Output Messages Graph Visualiser Notifications

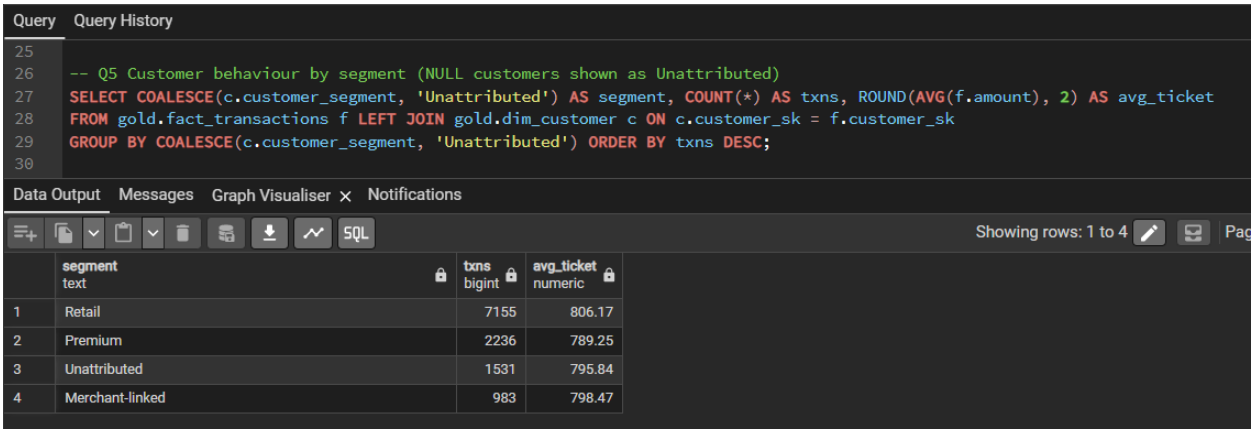
	channel_name text	row_count bigint	null_customer bigint	null_merchant bigint
1	WALLET	2900	35	887
2	UPI	6634	67	0
3	GATEWAY	2371	1429	0

15. SQL Scripts (Sprint 2)

Script	Purpose

Script	Purpose
sql/staging_ddl.sql	Creates the ten staging tables
sql/gold_ddl.sql	Creates the gold schema, generates dim_date, seeds dim_channel
sql/reporting_queries.sql	Business reporting queries
sql/backfill_gateway_customers.sql	Optional customer backfill for gateway rows
sql/reconciliation.sql	Builds gold.reconciliation_result (section 21)
sql/stream_ddl.sql	Stream tables for the real-time extension
python/db_connect.py, load_staging.py	Connection and staging load helpers
python/data_profiling.py, data_quality_checks.py	Profiling and the named quality checks
python/cleanse_transactions.py	Builds the silver layer and the rejected-row files
tests/test_warehouse.py	Automated warehouse checks

15.1 Reporting queries



Query | Query History

```sql
-- Q1 Success rate by channel (expect about 88% for every channel)
SELECT ch.channel_name, COUNT(*) AS total_txns,
  ROUND(100.0 * SUM(CASE WHEN f.status = 'SUCCESS' THEN 1 ELSE 0 END) / COUNT(*), 2) AS success_rate_pct
FROM gold.fact_transactions f
JOIN gold.dim_channel ch ON ch.channel_sk = f.channel_sk
GROUP BY ch.channel_name ORDER BY total_txns DESC;
```

Data Output | Messages | Graph Visualiser | Notifications

Showing rows: 1 to 3

	channel_name text	total_txns bigint	success_rate_pct numeric
1	UPI	6634	87.43
2	WALLET	2900	87.72
3	GATEWAY	2371	87.09

Query | Query History

```sql
-- Q3 Top 10 merchants by successful value
SELECT m.merchant_id, m.merchant_name, m.category, COUNT(*) AS txns, ROUND(SUM(f.amount), 2) AS total_value
FROM gold.fact_transactions f JOIN gold.dim_merchant m ON m.merchant_sk = f.merchant_sk
WHERE f.status = 'SUCCESS'
GROUP BY m.merchant_id, m.merchant_name, m.category ORDER BY total_value DESC LIMIT 10;
```

Data Output | Messages | Graph Visualiser | Notifications

Showing rows: 1 to 10

	merchant_id text	merchant_name text	category text	txns bigint	total_value numeric
1	MER0003	Bhatti And Sons	Entertainm...	83	89600.30
2	MER0128	Nath Plc	Other	67	79950.68
3	MER0008	Parikh, Raja And Sem	Fuel	77	77470.68
4	MER0102	Sood Ltd	Shopping	65	75970.89
5	MER0040	Venkatesh And Sons	Entertainm...	63	74299.89
6	MER0036	Mohanty Ltd	Transport	87	73412.97
7	MER0072	Bobal-Dubey	Utilities	75	72786.58
8	MER0011	Char Llc	Food & Dini...	79	70707.62
9	MER0041	Dhaliwal, Agrawal And Sekhon	Fuel	76	69943.08
10	MER0077	Narayan, Tripathi And Raju	Food & Dini...	72	69896.60

Query | Query History

```sql
-- Q2 Monthly volume and value, successful transactions
SELECT d.year, d.month, d.month_name, COUNT(*) AS txns, ROUND(SUM(f.amount), 2) AS total_value
FROM gold.fact_transactions f JOIN gold.dim_date d ON d.date_sk = f.date_sk
WHERE f.status = 'SUCCESS'
GROUP BY d.year, d.month, d.month_name ORDER BY d.year, d.month;
```

Data Output | Messages | Graph Visualiser | Notifications

Showing rows: 1 to 6

	year integer	month integer	month_name text	txns bigint	total_value numeric
1	2025	1	January	1799	1455020.09
2	2025	2	February	1590	1279202.88
3	2025	3	March	1815	1491145.15
4	2025	4	April	1714	1315023.27
5	2025	5	May	1783	1408077.58
6	2025	6	June	1708	1404123.30

Query | Query History

```sql
-- Q4 Category performance
SELECT m.category, COUNT(*) AS txns, ROUND(AVG(f.amount), 2) AS avg_ticket,
  ROUND(100.0 * SUM(CASE WHEN f.status = 'FAILED' THEN 1 ELSE 0 END) / COUNT(*), 2) AS failure_rate_pct
FROM gold.fact_transactions f JOIN gold.dim_merchant m ON m.merchant_sk = f.merchant_sk
GROUP BY m.category ORDER BY txns DESC;
```

Data Output | Messages | Graph Visualiser | Notifications

Showing rows: 1 to 10

	category text	txns bigint	avg_ticket numeric	failure_rate_pct numeric
1	Entertainm...	1452	823.92	9.02
2	Healthcare	1212	765.13	10.15
3	Education	1187	847.22	8.26
4	Food & Dini...	1167	818.87	8.05
5	Grocery	1114	749.70	9.43
6	Utilities	1088	756.55	9.38
7	Fuel	1033	804.67	9.58
8	Other	983	848.42	9.26
9	Shopping	934	772.47	9.85
10	Transport	848	807.63	8.61

Query | Query History

```sql
-- Q6 Fraud rate by month among scored transactions
SELECT d.month, d.month_name, COUNT(f.is_fraud) AS scored_txns, SUM(f.is_fraud) AS flagged,
  ROUND(100.0 * SUM(f.is_fraud) / NULLIF(COUNT(f.is_fraud), 0), 2) AS fraud_rate_pct
FROM gold.fact_transactions f JOIN gold.dim_date d ON d.date_sk = f.date_sk
GROUP BY d.month, d.month_name ORDER BY d.month;
```

Data Output | Messages | Graph Visualiser | Notifications

Showing rows: 1 to 6

	month integer	month_name text	scored_txns bigint	flagged bigint	fraud_rate_pct numeric
1	1	January	162	1	0.62
2	2	February	151	1	0.66
3	3	March	180	0	0.00
4	4	April	138	0	0.00
5	5	May	176	0	0.00
6	6	June	161	0	0.00

15.2 Optional gateway customer backfill

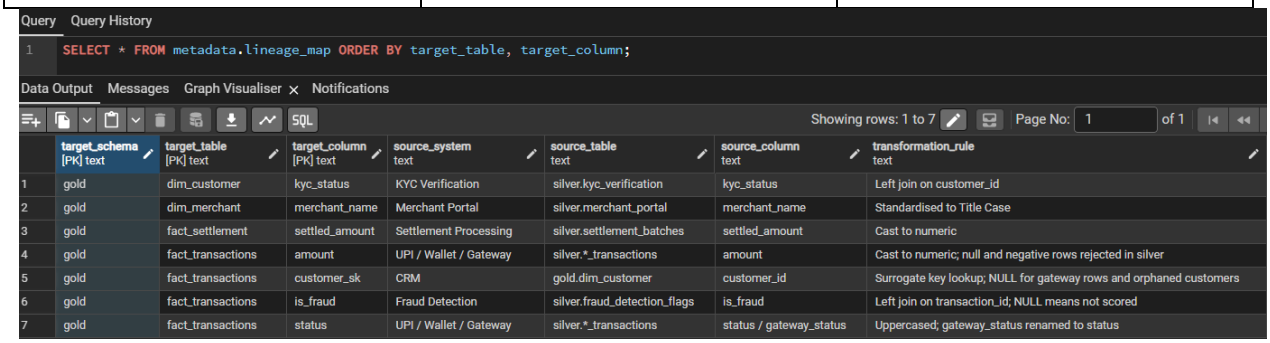
Fills customer_sk for gateway rows from the customer on a linked mobile banking session. This is an inference that the session user is the payer, and it recovered 942 of 2,371 gateway rows. It must run after the fact load, because the fact load rebuilds the table.

```
UPDATE gold.fact_transactions f
SET customer_sk = c.customer_sk
FROM (SELECT DISTINCT ON (transaction_id) transaction_id, customer_id
      FROM silver.mobile_banking_sessions
      WHERE NULLIF(transaction_id, '') IS NOT NULL
      ORDER BY transaction_id, login_timestamp) s
JOIN gold.dim_customer c ON c.customer_id = s.customer_id
WHERE f.transaction_id = s.transaction_id AND f.customer_sk IS NULL;
WHERE f.transaction_id = s.transaction_id AND f.customer_sk IS NULL;
```

16. Source-to-Target Mapping (Sprint 3)

Target column	Source	Rule
fact_transactions.transaction_id	transaction_id in UPI, wallet, gateway	Direct copy; unique
fact_transactions.customer_sk	dim_customer via customer_id	LEFT join; NULL for gateway rows and missing customers
fact_transactions.merchant_sk	dim_merchant via merchant_id	LEFT join; NULL for peer-to-peer transfers
fact_transactions.date_sk	dim_date via the date of the transaction timestamp	Text cast to timestamp, then to date
fact_transactions.channel_sk	dim_channel	UPI, WALLET or GATEWAY by source table
fact_transactions.amount	amount	Cast to numeric; null and negative rows rejected in silver
fact_transactions.status	status, or gateway_status for the gateway	Uppercased
fact_transactions.is_fraud, fraud_score	fraud_detection_flags	LEFT join on transaction_id after deduplication; NULL means not scored
dim_customer.name, gender, city, state, customer_segment	CRM	Direct copy
dim_customer.kyc_status, risk_score	KYC	LEFT join on customer_id

Target column	Source	Rule
dim_merchant.merchant_name	Merchant Portal	Standardised to Title Case
dim_merchant.category, city, state, settlement_cycle	Merchant Portal	Direct copy
fact_settlement.settlement_id	Settlement	Direct copy
fact_settlement.merchant_sk	dim_merchant via merchant_id	Surrogate key lookup
fact_settlement.date_sk	dim_date via batch_date	Surrogate key lookup
fact_settlement.total_amount, settled_amount	Settlement	Cast to numeric
fact_settlement.settlement_status	Settlement	Direct copy
dim_date (all columns)	Generated	generate_series over 2025
dim_channel (all columns)	Seeded	Three fixed rows



Query Query History

```
SELECT * FROM metadata.lineage_map ORDER BY target_table, target_column;
```

Data Output Messages Graph Visualiser Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	target_schema [PK] text	target_table [PK] text	target_column [PK] text	source_system text	source_table text	source_column text	transformation_rule text
1	gold	dim_customer	kyc_status	KYC Verification	silver.kyc_verification	kyc_status	Left join on customer_id
2	gold	dim_merchant	merchant_name	Merchant Portal	silver.merchant_portal	merchant_name	Standardised to Title Case
3	gold	fact_settlement	settled_amount	Settlement Processing	silver.settlement_batches	settled_amount	Cast to numeric
4	gold	fact_transactions	amount	UPI / Wallet / Gateway	silver.*_transactions	amount	Cast to numeric; null and negative rows rejected in silver
5	gold	fact_transactions	customer_sk	CRM	gold.dim_customer	customer_id	Surrogate key lookup; NULL for gateway rows and orphaned customers
6	gold	fact_transactions	is_fraud	Fraud Detection	silver.fraud_detection_flags	is_fraud	Left join on transaction_id; NULL means not scored
7	gold	fact_transactions	status	UPI / Wallet / Gateway	silver.*_transactions	status / gateway_status	Uppercased; gateway_status renamed to status

17. Data Lineage Document (Sprint 3)

Source	Bronze	Silver	Gold	Consumed by
UPI	staging.upi_transactions	silver.upi_transactions (status uppercased; duplicates, nulls and negatives rejected)	fact_transactions, channel UPI	Dashboards, reporting queries
Wallet	staging.digital_wallet_transactions	silver.digital_wallet_transactions (same rules)	fact_transactions, channel WALLET	Dashboards
Gateway	staging.payment_gateway_transacti	silver.payment_gateway_transactio	fact_transactions, channel	Dashboards

Source	Bronze	Silver	Gold	Consumed by
	ons	ns (gateway_status uppercased)	GATEWAY	
Merchant Portal	staging.merchant_portal	silver.merchant_portal (Title Case names)	dim_merchant	Merchant views
CRM	staging.crm_customers	silver.crm_customers	dim_customer	Customer views
KYC	staging.kyc_verification	silver.kyc_verification	dim_customer (kyc_status, risk_score)	Customer views
Settlement	staging.settlement_batches	silver.settlement_batches	fact_settlement	Reconciliation
Fraud Detection	staging.fraud_detection_flags	silver.fraud_detection_flags	fact_transactions (is_fraud, fraud_score)	Fraud views
Mobile Banking	staging.mobile_banking_sessions	silver.mobile_banking_sessions	Not in the star schema	Gateway customer backfill, audit
API Gateway Logs	staging.api_gateway_logs	silver.api_gateway_logs	Not in the star schema	Technical audit trail
Event stream (extension)	staging.stream_events	Planned	Planned	Live view

Mobile banking and API logs stop at silver by design: they support audit and investigation but are not analytical facts.

Governance. metadata.lineage_map is the authoritative, queryable lineage. Any change to a transformation, script or gold column must update the map and this section before it counts as done.

18. Metadata Repository (Sprint 3)

Object	Purpose
metadata.table_catalog	Layer, table, business description, row count, owner
metadata.column_catalog	Column, type, business definition, PII flag
metadata.lineage_map	Source-to-target mapping, one row per gold

Object	Purpose
	column
metadata.job_log	Run history written by the Pentaho job
metadata.stream_watermark	Progress marker for the streaming extension

Personal data columns: name, dob, phone, email (CRM), upi_vpa (UPI), bank_account (Merchant Portal) and client_ip (API logs).

19. FinTech Business Glossary (Sprint 3)

Term	Definition
UPI	Unified Payments Interface, India's real-time interbank payment system
VPA	Virtual Payment Address, a UPI ID used instead of bank account details
Digital wallet	A stored-value account used for payments without touching a bank account
Payment gateway	The layer that authorises and processes card, UPI and net-banking payments
Merchant	A business registered to accept digital payments
Settlement	Transfer of collected funds to a merchant's bank account
Settlement cycle	When a merchant is paid: T+1, T+2 or weekly
Reconciliation	Matching transaction totals against the amounts settled
Chargeback	A transaction reversed after a dispute
KYC	Know Your Customer, identity verification before service is enabled
Fraud score	A model probability (0 to 1) that a transaction is fraudulent
Success rate	Share of attempted transactions that complete successfully
Customer segment	A grouping of customers such as Retail or Premium
API gateway	The entry point that routes and logs API requests

Term	Definition
Bronze, silver, gold	Raw, cleansed and business-ready data layers
ETL	Extract, transform, load
Star schema	A central fact table surrounded by dimension tables
Fact and dimension	Measurable events versus the descriptive context around them
Surrogate and natural key	A warehouse-generated key versus the source system's own identifier
Grain	What one row of a fact table represents
Data lineage	The documented path of data from source to report
Source-to-target mapping	Field-level rules linking source fields to target fields
Data profiling	Examining data for structure, nulls and anomalies
PII	Personally identifiable information
Orchestration	Automatic ordering and monitoring of ETL steps
Orphaned record	A record whose referenced key has no match in the master table
Event time and ingestion time	When something happened versus when the platform stored it
Micro-batch	Processing a stream in small, frequent batches

20. Pentaho Jobs (Sprint 3)

The master job pentaho/jobs/eftp_master_pipeline.kjb runs the whole pipeline in order and stops on the first failure.

Order	Entry	Runs
1	Start	
2	Job	run_all_ingestion.kjb (the ten ingestion transformations)

Order	Entry	Runs
3	Shell	python python/cleanse_transactions.py , working directory at the project root
4	Transformation	load_dim (dim_customer and dim_merchant)
5	Transformation	load_fact_transactions
6	Transformation	load_fact_settlement
7	Execute SQL script	sql/backfill_gateway_customers .sql (optional)
8	Execute SQL script	sql/reconciliation.sql
9	Shell	python -m pytest tests -q
10	Write to log	Pipeline completed successfully

Every step is joined by a success hop. A failure hop from each entry goes to a Write to log entry and then an Abort job entry, so a failed step halts the run and records why.

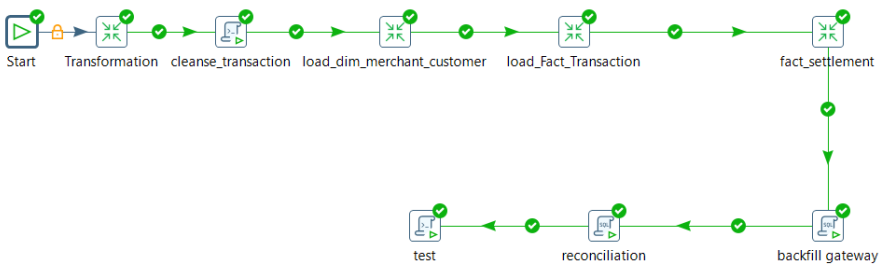
Design rules. Dimensions load before facts because facts need their surrogate keys. Paths use the variable PROJECT_HOME, defined in kettle.properties inside the user's .kettle folder (C:\Users\name.kettle on Windows, ~/.kettle on Linux or macOS), and Spoon is restarted after editing it. Reruns are safe: staging truncates, silver tables are replaced, dimensions update in place, and fact tables truncate and reload.

Audit logging. The job's Log tab writes every run to metadata.job_log (start, end, status, rows read and written, errors).

Command-line run (proves it works without Spoon):

```
Kitchen.bat /file:%PROJECT_HOME%\pentaho\jobs\eftp_master_pipeline.kjb /level:Basic
kitchen.sh -file=$PROJECT_HOME/pentaho/jobs/eftp_master_pipeline.kjb -level=Basic
```

The first form is for Windows, the second for Linux and macOS. A scheduler (Task Scheduler or cron) can call the same command.



21. Validation Report (Sprint 3)

21.1 Scope

Two validations are reported: warehouse integrity (the PASS checks in section 14) and settlement reconciliation (this section).

21.2 Method

Level	Compares	Expected difference
1	Warehouse total of successful transactions per merchant and week against the settlement file's total_amount	Small, because the settlement file was built before cleansing removed duplicate and negative rows
2	total_amount against settled_amount	The payout deduction, 0 to 3% in this data

Week rule. Settlement batches are dated on the Sunday that ends a Monday-to-Sunday week. A transaction maps to its batch with full_date + ((7 - day of week) mod 7) days, so a Sunday transaction belongs to the batch of that same Sunday.

Tolerance. Level 1 uses 1% of total_amount as a starting threshold. Choose the final threshold from the distribution below and state the reason in the results.

21.3 Reconciliation query

```
DROP TABLE IF EXISTS gold.reconciliation_result;

CREATE TABLE gold.reconciliation_result AS
WITH txn_week AS (
  SELECT merchant_id, batch_date, COUNT(*) AS txn_count, SUM(amount) AS
txn_total
  FROM (
    SELECT m.merchant_id, f.amount,
           d.full_date + ((7 - EXTRACT(DOW FROM d.full_date)::int) % 7) AS
batch_date
    FROM gold.fact_transactions f
    JOIN gold.dim_merchant m ON m.merchant_sk = f.merchant_sk
    JOIN gold.dim_date d     ON d.date_sk     = f.date_sk
    WHERE f.status = 'SUCCESS'
  ) x
  GROUP BY merchant_id, batch_date
),
settle AS (
  SELECT m.merchant_id, d.full_date AS batch_date, s.settlement_id,
s.total_amount, s.settled_amount
  FROM gold.fact_settlement s
  JOIN gold.dim_merchant m ON m.merchant_sk = s.merchant_sk
  JOIN gold.dim_date d     ON d.date_sk     = s.date_sk
```

```
)
SELECT COALESCE(s.merchant_id, t.merchant_id) AS merchant_id,
       COALESCE(s.batch_date, t.batch_date)   AS batch_date,
       s.settlement_id, t.txn_count, t.txn_total, s.total_amount,
s.settled_amount,
       t.txn_total - s.total_amount AS level1_diff,
       100.0 * (s.total_amount - s.settled_amount) / NULLIF(s.total_amount,
0) AS level2_shortfall_pct,
       CASE
         WHEN s.settlement_id IS NULL THEN 'MISSING_SETTLEMENT'
         WHEN t.txn_total IS NULL THEN 'MISSING_TRANSACTIONS'
         WHEN ABS(t.txn_total - s.total_amount) <= 0.01 * s.total_amount THEN
'WITHIN_TOLERANCE'
         ELSE 'MISMATCH'
       END AS recon_status
FROM settle s
FULL OUTER JOIN txn_week t ON t.merchant_id = s.merchant_id AND t.batch_date
= s.batch_date;
FULL OUTER JOIN txn_week t ON t.merchant_id = s.merchant_id AND t.batch_date =
s.batch_date;
```

21.4 Result queries

```
-- Status counts
SELECT recon_status, COUNT(*) AS merchant_weeks FROM gold.reconciliation_result
GROUP BY recon_status ORDER BY 2 DESC;

-- Payout shortfall (Level 2), expect under 3%
SELECT COUNT(*) AS batches, ROUND(AVG(level2_shortfall_pct), 2) AS avg_shortfall_pct,
       ROUND(MAX(level2_shortfall_pct), 2) AS max_shortfall_pct
FROM gold.reconciliation_result WHERE settlement_id IS NOT NULL;

-- Distribution of Level 1 differences, to justify the tolerance
SELECT CASE WHEN p < 0.5 THEN 'under 0.5%' WHEN p < 1 THEN '0.5 to 1%'
            WHEN p < 5 THEN '1 to 5%' ELSE '5% and above' END AS band, COUNT(*) AS
merchant_weeks
FROM (SELECT 100.0 * ABS(level1_diff) / NULLIF(total_amount, 0) AS p
      FROM gold.reconciliation_result
      WHERE settlement_id IS NOT NULL AND txn_total IS NOT NULL) x
GROUP BY 1 ORDER BY 1;

-- Ten largest mismatches
SELECT merchant_id, batch_date, txn_total, total_amount, ROUND(level1_diff, 2) AS
diff
FROM gold.reconciliation_result WHERE recon_status = 'MISMATCH'
ORDER BY ABS(level1_diff) DESC LIMIT 10;
ORDER BY ABS(level1_diff) DESC LIMIT 10;
```

21.5 Results recorded

Check	Result	Comment

Check	Result	Comment
Merchant-weeks reconciled	3864	
WITHIN_TOLERANCE	3440	Most rows are expected here
MISMATCH	120	Explain the largest by merchant and week
MISSING_SETTLEMENT or MISSING_TRANSACTIONS	304	Should be 0; a larger number means a week-alignment or join problem to fix first
Average and maximum Level 2 shortfall	1.52	Expected 0 to 3%
Warehouse integrity checks	Complete	All PASS (section 14)
Tolerance chosen and reason	0.5%	Based on the distribution above

22. Power BI Dashboard (Sprint 3)

22.1 Model

Data is imported from PostgreSQL (database eftp_db) from fact_transactions, fact_settlement, reconciliation_result, dim_customer, dim_merchant, dim_date and dim_channel. Every relationship is one-to-many from the dimension to the fact, single direction, on the surrogate keys (date_sk, merchant_sk, customer_sk, channel_sk). dim_date is marked as the date table using full_date. Key and versioning columns are hidden.

22.2 Measures

```
Total Transactions = COUNTROWS(fact_transactions)
Successful Transactions = CALCULATE([Total Transactions], fact_transactions[status] =
"SUCCE _amount])) SS")
Success Rate = DIVIDE([Successful Transactions], [Total Transactions])
Failure Rate = DIVIDE(CALCULATE([Total Transactions], fact_transactions[status] =
"FAILED"), [Total Transactions])
Total Value = CALCULATE(SUM(fact_transactions[amount]), fact_transactions[status] =
"SUCCESS")
Avg Ticket = DIVIDE([Total Value], [Successful Transactions])
Scored Transactions = COUNT(fact_transactions[is_fraud])
Fraud Rate Scored = DIVIDE(SUM(fact_transactions[is_fraud]), [Scored Transactions])
Unattributed Transactions = CALCULATE([Total Transactions],
fact_transactions[customer_sk] = BLANK())
Settlement Shortfall Pct = DIVIDE(SUM(fact_settlement[total_amount]) -
SUM(fact_settlement[settled_amount]), SUM(fact_settlement[total
```

22.3 Pages

Page	Visuals	Business question

Page	Visuals	Business question
Executive Summary	KPI cards: transactions, value, success rate, fraud rate, shortfall	How is the business performing?
Transaction Trends	Volume and value by month or day, channel slicer	What are the payment trends?
Success and Failure	Success and failure rate by channel, failure counts	Where are payments failing?
Merchant Performance	Top 10 merchants by value, category breakdown	Which merchants perform best?
Customer Behaviour	Transactions and average ticket by segment, KYC split	How do customers pay?
Fraud Monitoring	Scored, flagged and rate by month, high-score table	Is fraud rising?
Settlement and Reconciliation	Status counts, shortfall distribution, mismatch table	Do payouts match transactions?

22.4 Data caveats shown on the dashboard

- Transactions without a customer are shown as Unattributed: 1,429 gateway rows (12.0% of all transactions), plus 102 UPI and wallet rows whose customer is missing from the CRM.
- Only about 8% of transactions are fraud-scored, and the fraud rate is calculated over scored transactions only. Confirmed fraud is rare in this data, so monthly rates are based on small counts.

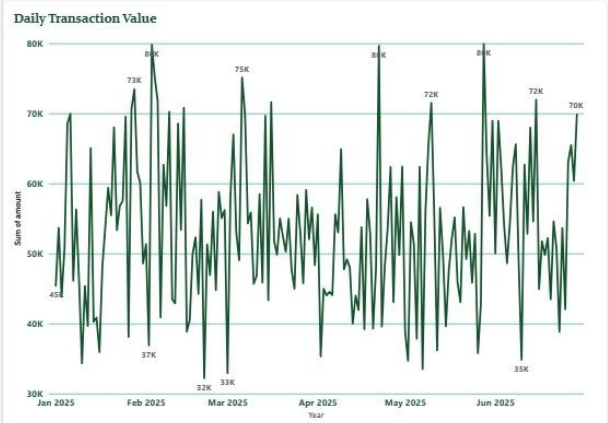
FinTech Transaction Analytics Dashboard

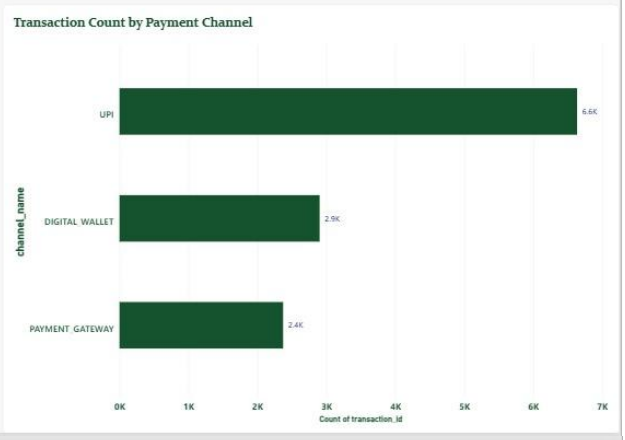
Total Transaction Value
$9.54M

Total Transactions
12K

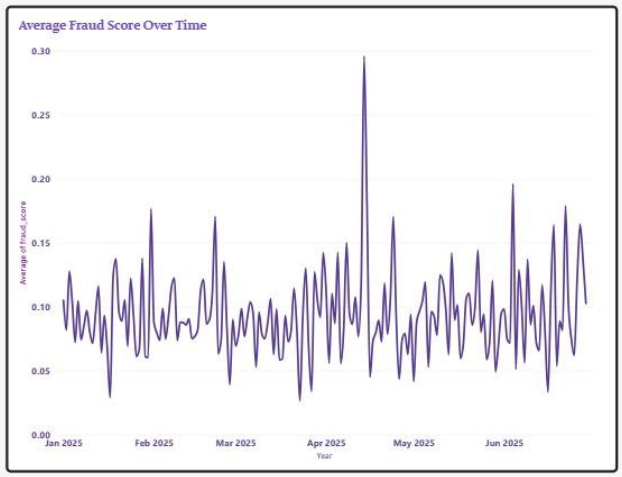
Successful Transactions
10K

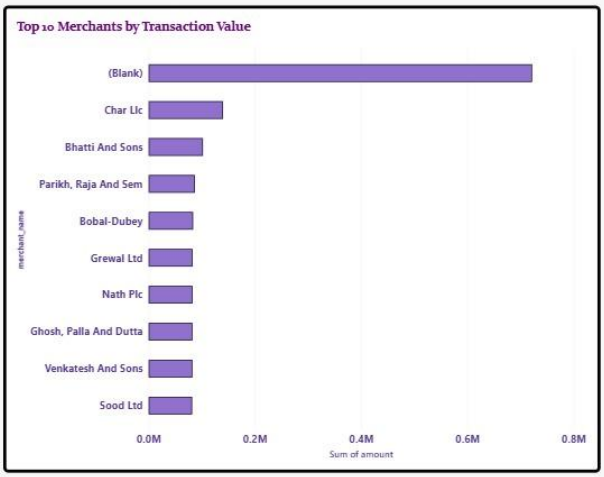
Success Rate
87.43%





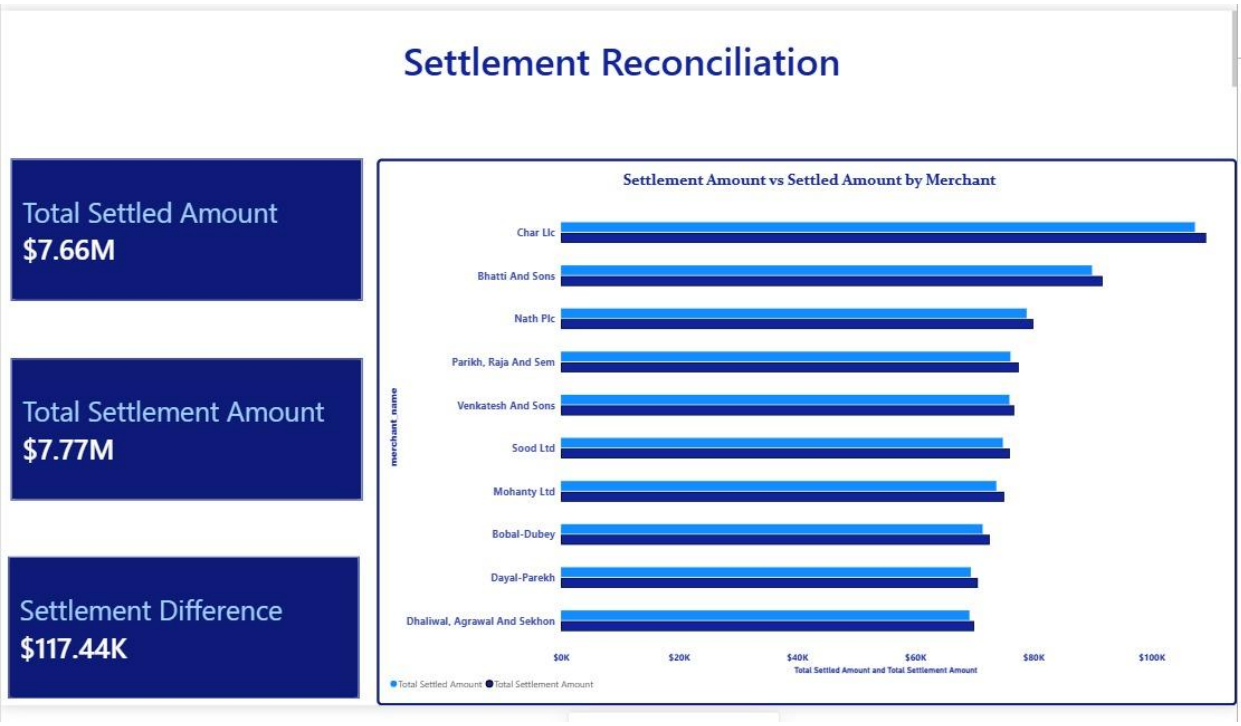
Merchant & Fraud Analytics





Fraudulent Transactions
2

Fraud Rate
0.02%



23. Real-Time Ingestion Extension

This extension goes beyond the brief. A Python generator publishes about five payment events per second to a Kafka topic, and a Pentaho Kafka Consumer step lands them in staging.stream_events. Delivery is near real time (micro-batch), not event-at-a-time.

```
event_generator.py -> Kafka topic eftp.transactions -> Pentaho Kafka Consumer
  -> sub-transformation (Get records from stream) -> Table output ->
staging.stream_events
```

Item	Detail
Event fields	event_id, transaction_id (TXS prefix), channel, customer_id, merchant_id, amount, status, event_ts
Injected faults (about 3%)	Duplicate delivery 1.0%, null amount 0.5%, negative amount 0.5%, status casing 0.5%, late event 0.5%
Bronze table	payload as TEXT, kafka_offset, kafka_ts (epoch milliseconds), ingested_at defaulting to now()
Consumer settings	Batch of 100 records or 5,000 ms, commit size 100, offsets committed on batch completion, auto.offset.reset earliest

Item	Detail
Time zones	event_ts is local time without an offset (Indian Standard Time); kafka_ts is UTC
Status	Ingestion built. Silver cleansing and the gold upsert are not yet built

24. Definition of Done Verification

Criterion	Evidence	Section
Business requirements are addressed	FR-01 to FR-11 mapped to delivered work in the deliverable index	1, 2
ETL pipelines execute successfully	Master job run and job_log rows	20
Data quality checks are documented	Data Quality Report with counts and queries	12
PostgreSQL tables are populated correctly	Table counts and PASS validation	14
Source-to-target mappings are updated	Mapping table and metadata.lineage_map	16, 18
Code is committed to Git with meaningful messages	git log output	6
Documentation is completed	This report and the companion files	All
Sprint review presentation is delivered	Slide outline and deck	25

Tick each line only after attaching the evidence. Complete the same check at the end of every sprint.

25. Sprint Presentation Outline

Slide	Content	Source in this report
1	Title, team, repository link	Title
2	Business problem and objectives	2
3	Architecture	4

Slide	Content	Source in this report
4	Sources and formats	7
5	Ingestion, logging and error handling	9
6	Data quality findings and fixes	12
7	Star schema	13
8	Metadata, lineage and glossary	16 to 19
9	Orchestrated pipeline run	20
10	Reconciliation results	21
11	Dashboard demonstration	22
12	Real-time extension, challenges, future work	23, 26, 28

Rehearse a live run of the job, and keep screenshots as a backup.

26. Challenges and Resolutions

Challenge	Cause	Resolution
ydata-profiling would not install	It does not support Python 3.13 and later	Virtual environment with Python 3.10 or 3.11, or the pandas fallback built into the script
Dimension step failed on its sequence	The sequence name included the schema, and Pentaho added the schema again	Bare sequence name, with the target schema set separately
Dimension step listed an unexpected field (load_date)	The column exists only in the gold table	Removed it from the field grid so the database default applies
Gateway rows had no customer	The source has no customer_id	NULL customer_sk, backfill of 942 rows from mobile sessions, documented
fact_settlement load failed	Target table name was misspelled	Table chosen with Browse; the stray table dropped
Silver tables for mobile banking and API logs missing	They had not been promoted from staging	Created from staging; lineage updated
Reconciliation query failed on	Column absent from the	Column removed from the

Challenge	Cause	Resolution
total_transactions	deployed fact_settlement	query; week rule corrected
A one rupee settlement tolerance flagged nearly every batch	Every payout is 0 to 3% below the total	Percentage-based threshold
Stream load failed: jsonb against string	Pentaho sends the message as a string	payload stored as TEXT, parsed in silver
Stream load failed: bigint into a timestamp column	Kafka timestamp mapped to ingested_at	New kafka_ts column; ingested_at left to its default
Watermark table failed: schema missing	The metadata schema is created in Sprint 3	CREATE SCHEMA IF NOT EXISTS metadata

27. Learning Outcomes

Outcome	Evidence	Section
Analyze and scope enterprise FinTech data engineering requirements	BRD, charter, backlog	2, 3, 5
Design and implement end-to-end ETL pipelines using Pentaho	Ingestion, load and master job	9, 20
Integrate heterogeneous financial sources	Ten sources in five formats	7, 8
Profile data and assess quality with Python	Profiling and quality reports	11, 12
Design and populate a PostgreSQL warehouse by dimensional modelling	Star schema and warehouse	13, 14
Document metadata and lineage for governance	Mapping, lineage, catalog, glossary	16 to 19
Manage artefacts and collaborate with Git and GitHub	Repository, history, release tag	6
Build executive dashboards for payment analytics	Power BI report	22
Present an enterprise-grade solution using Agile sprint	Backlog, sprint reviews, presentation	5, 25

Outcome	Evidence	Section
practices		

28. Limitations and Future Work

Limitations. The data is synthetic, and the platform runs on one machine. Gateway transactions cannot be attributed to a customer without an inferred backfill. Confirmed fraud is rare in the source, so fraud analytics rest on few positives. Dimensions are Type 1 only. The reconciliation tolerance is chosen by the author, not agreed with a business owner. The streaming extension covers ingestion only, and it is micro-batch rather than true streaming.

Future work. Slowly changing dimensions (Type 2); scheduled runs; automated data quality alerts; stream silver and gold stages using the watermark table; incremental instead of full fact loads; access control for PII columns; a payer identifier in the gateway feed; a data quality dashboard.

29. Digital Assignment Details and Submission

Sno	Digital assignment	Sprints	Report sections	Status
1	DA 1	Sprint 0 and Sprint 1	2 to 10	Submitted / verified
2	DA 2	Sprint 2	11 to 15	Submitted / verified
3	DA 3	Sprint 3	16 to 25 and the final release	Submitted / verified